HARDWARE READING MINDSET
Read every SystemVerilog line as hardware, not software. continuous assign / always_comb becomes gates, muxes, comparators, adders, subtractors, shifters, etc. always_ff on a clock edge becomes flops/registers. A variable declared logic/reg is not storage by itself; storage appears only when the behavior requires remembering a past value. Initial synthesis structure is close to what you coded; later optimization may share, duplicate, resize, or remove logic.

PROCEDURAL BLOCKS
always_comb: all outputs should get values on every path; missing path means latch. It auto-sensitizes, runs at time 0, and gives better tool checking than always @*. always_ff: use for flop/register behavior; nonblocking <=. Reset in sensitivity list = asynchronous reset, reset checked inside posedge-only block = synchronous reset. always_latch is not magic; it documents intentional latch inference and lets tools check that latch-like behavior is actually described.

LATCH / FSM STYLE
Safe FSM pattern: typedef enum state_t; always_ff state <= nxt_state; always_comb begin default outputs; nxt_state = state; case(state) ... endcase end. Defaults prevent accidental latches. A clocked block with no assignment just holds the flop value, which is normal. A combinational block with no assignment on a path must remember, which creates a latch. Use default/safe state for recovery; unique/priority only documents intent, it does not fill missing assignments.

CODE -> CIRCUIT PATTERNS
if/else becomes mux or priority logic; ternary ?: is a 2:1 mux; case is mux/decode; == is equality tree; < compares/subtracts; + - * infer arithmetic blocks. Fixed for loop is unrolled into replicated hardware; the loop index is not hardware. Practice ssum: each a[i] is AND/masked with mask[i], then selected terms feed an adder chain/tree producing 6-bit sum.

WIDTHS / SIGN / ARRAYS
Packed dimensions are bits of one signal: logic [7:0] x. Unpacked dimensions are collections: logic [7:0] mem[0:15] = sixteen bytes. input [3:0] a[0:2] = array of three 4-bit vectors. Arithmetic width/sign matters: N+N may need N+1; signed math only behaves signed if operands/casts are signed. concat/part-select are wiring operations. Reduction operators (& | ^) collapse a vector to one bit.

TASKS VS FUNCTIONS
Function: zero simulation time; no @, #, wait, event/timing control; returns one value or void; useful for pure calculations and can synthesize as inlined combinational logic when clean. Task: may consume time; can contain waits, delays, fork/join, repeated clock waits; best for testbench actions such as reset, send command, wait for response, check protocol. Function can call functions, not tasks. Task can call both.

TASK ARGUMENT MODES
input argument is copied at call time; if the task waits, the input copy will not track later changes. output/inout copy a result back when task returns. ref passes a handle to the actual variable, so a task waiting over time can see clk/done/data changes live. Use ref for signals monitored across time, especially clk in wait loops. ref tasks should be automatic and the actual must be a variable, not a plain net.

TESTBENCH BASICS
initial reset sequence, forever clock generator, then stimulus/check tasks. repeat(N) @(posedge clk) waits N clocks; repeat count is sampled when loop begins. wait(cond) waits level; @(posedge sig) waits event. Self-check with assert(expr) else $fatal/$stop; use !==/=== when X/Z matters. Use small tasks like reset_dut, send_spi, wait_done, check_data instead of one giant initial block.

FORK / JOIN / TIMEOUT
begin/end executes sequentially. fork/join starts branches in parallel and join waits until all finish; timing in each branch is relative to fork entry. Named blocks can be disabled, like breaking out of a wait. Timeout idiom: fork one branch waits for done and checks result; second branch waits N clocks then $fatal; success branch disables the timeout block. Useful because a frozen testbench is worse than a failing test.

ASSERTIONS
Immediate assertions check now: assert(result==expected) else $fatal. Concurrent assertions check behavior across clock cycles: assert property(@(posedge clk) disable iff(!rst_n) req |-> ##[1:3] ack). Good for protocols: request followed by ack, signal stays high until done, no illegal transition, autozero high for first 100 conversions. Assertions/coverage are verification constructs, not normal synthesized datapath logic.

RANDOM / CLASSES / UVM
rand/randc live inside classes; call assert(obj.randomize()). Constraints restrict legal values with inside, if, dist, relational expressions. randc cycles through allowed values before repeat. Constrained random verification only helps if you also have a scoreboard/reference model to self-check. Class = properties + methods + constructor new(); extends gives inheritance. UVM is reusable object-oriented verification using transactions, sequences, drivers, monitors, scoreboards.

INTERFACES / PACKAGES
interface bundles related pins and tasks/functions; a bus functional model (BFM) is usually an interface plus tasks to drive and monitor a bus. package holds typedefs, enums, parameters, functions/tasks shared by modules or TBs; import pkg::* makes names visible. `include textually pastes code; package is a compiled namespace.
Interfaces/classes/packages are mostly verification/organization tools, not a reason to invent complicated DUT hardware.

LOOPS / GENERATE
Synthesis loop rule: fixed/static iteration count can be unrolled; data-dependent unbounded loops are not synthesizable as simple combinational logic. A parameter bound is okay because it is known at elaboration. generate for/if/case runs before hardware exists; genvar is elaboration-only. Move invariant expressions out of loops or you replicate expensive arithmetic. Loop with internal @(posedge clk) describes sequential behavior over cycles, often as FSM-like hardware.

PARAMETER VS INPUT
Parameter/localparam is compile-time configuration: vector widths, array sizes, FAST_SIM, architecture choice, generate branch selection. Unused parameter branches can disappear completely. Input port is runtime data: changes while circuit operates and cannot size hardware or be optimized away as a constant. Exam phrase: parameter configures what hardware is built; input controls behavior of already-built hardware.

SYNTHESIS FLOW
Flow: parse syntax/semantics -> policy check synthesizable subset -> elaborate/translate -> architectural optimization -> multilevel Boolean optimization -> technology mapping -> gate netlist. Elaboration unrolls loops, substitutes macros/parameters, evaluates generate and constant functions. Architecture optimization chooses adders/multipliers/sharing/pipelining structures. Mapping selects cells from the library. Good HDL exposes recognizable structures; bad HDL hides intent as giant random logic.

OPTIMIZATION PRIORITY
Architectural choices dominate area/speed: ripple vs carry-lookahead, pipeline stage placement, shared vs duplicated arithmetic, counter incrementer vs full adder, algebraic rearrangement, state encoding. Gate-level optimization then factors logic, removes constants/dead logic, uses don't-cares, resizes cells, inserts buffers, duplicates logic for timing, and fixes design rules. Timing/design-rule goals generally outrank area; area cleanup happens last.

RESOURCE SHARING
Before optimization, a case/if with A+B, A-B, B-A may show multiple arithmetic blocks after read/elaboration. Final synthesis may share one adder/subtractor plus muxes because the operations are mutually exclusive. Sharing saves area but can make the selected datapath/control path slower. Unsharing/duplication costs area but can shorten a critical path. Always separate 'arithmetic blocks inferred initially' from 'blocks actually used after optimization.'

LATE ARRIVING SIGNALS
Rule: push the late signal as close to the output as possible. If sub/control is late, compute A+B and A-B early in parallel, then use sub only as final mux select. If A/data is late, precompute everything independent of A, then feed A into the final arithmetic stage. Code style matters: algebraic reassociation and precomputed wires can force the structure you want instead of accepting the default shared adder/subtractor.

STA DEFINITIONS
clk2q: clock edge to Q valid. setup: D must be valid before capture edge. hold: D must remain stable after capture edge. max path is worst/slow propagation and checks setup: 'is circuit fast enough?' min path is fastest propagation and checks hold: 'is circuit too fast?' Positive slack means met. Back-to-back flops are the classic hold-risk topology because almost no logic exists between them.

STA FORMULAS
Max/setup slack = ClockPeriod - clk2q - TMAX - Tsetup - worst skew/uncertainty. Min/hold slack = clk2q + TMIN - Thold - worst skew/uncertainty. For max reports, slack is required - arrival. For min reports, slack is arrival - required. Clock uncertainty/skew always hurts because STA assumes worst case: launch edge late/early and capture edge early/late as needed to make margin smaller.

INPUT / OUTPUT DELAY
Input delay is not 'how much time DUT has.' It is how much of the cycle the previous chip/source stole before the DUT input becomes valid after the prior clock edge. So internal available time is reduced. Output delay is time before next edge that the next chip/receiver needs DUT output valid, so DUT must finish by Tclk - output_delay. These constraints model surrounding system timing at primary inputs/outputs.

FIXING TIMING
Setup fail: reduce combinational depth, restructure algebra, use faster cells/LVT, resize, duplicate/unshare, balance adder trees, pipeline/retime, or relax/declare correct false/multicycle paths. Hold fail: add delay/buffers or fix clock tree; slowing the clock period does not fix hold. Any timing fix can affect area/power and may create new min/max issues, so rerun both max and min reports.

PARASITICS / APR / SDF
Parasitic capacitance is unavoidable between conductors and slows switching. Modern processes worsen sidewall/neighbor capacitance because wires got tall and close, so RC delay/crosstalk matter more, not less. Wireload model is a pre-layout guess. APR places and routes cells; extractor computes real parasitics. SDF stores cell delays and interconnect flight times; back-annotate max for setup, min for hold in STA or ModelSim gate simulation.

FPGA / STD CELL / CUSTOM
FPGA: low NRE, reprogrammable, great for prototypes, but higher per-unit cost, slower, more power, LUT/routing overhead. Standard cell ASIC: high NRE/mask cost but faster, lower power, cheaper per unit at volume. Custom logic: maximum density/speed/power if hand-designed, but huge design time and painful changes. Microcontroller is another implementation choice when performance does not require custom digital hardware.

CLOCK GATING / GATE SIM
Clock gating lowers dynamic power when a block is idle, usually increases area slightly because gating cells/control are added, and should not change functional latency. Do not AND clocks like normal data; use proper latch/integrated clock-gating style so no glitches. Gate-level/post-synth sim failures often come from missing libraries, compiling RTL and netlist together, unreset Xs, TB sampling too early, latency from pipeline changes, or SDF/timing issues.

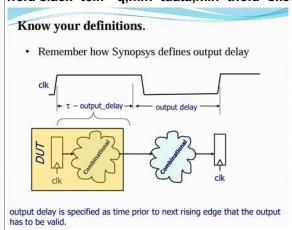
```
set TOP MazeRunner
set SRC_DIR ./ProjectDir
remove_design -all
# Libraries
set target_library [list ./saed32lvt_tt0p85v25c.db]
set link_library [list * ./saed32lvt_tt0p85v25c.db standard.sldb]

set file_list [glob -nocomplain ${SRC_DIR}/*.sv] # Analyze RTL, skip TB / sim-only files

foreach f $file_list {
    set base [file tail $f]
    if {[string match "*_tb.sv" $base]} { continue }
    if {[lsearch -exact $non_synth $base] >= 0} { continue }
    analyze -format sverilog $f
}

elaborate $TOP # Elaborate/link/check
current_design $TOP
link
check_design > check_design_precompile.txt

create_clock -name clk -period 2.75 -waveform {0 1.25} [get_ports clk] # Clock constraint
set_dont_touch_network [get_ports clk]
set_clock_uncertainty 0.125 [get_clocks clk]

set prim_inputs [remove_from_collection [all_inputs] [get_ports clk]] # Input constraints
set_input_delay -clock clk 0.6 $prim_inputs
set_driving_cell -lib_cell NAND2X1_LVT $prim_inputs
set_switching_activity -static_probability 0.5 -toggle_rate 0.25 \
    -base_clock clk $prim_inputs

set_output_delay -clock clk 0.5 [all_outputs]
# Output / design constraints

set_load 0.1 [all_outputs]
set_max_transition 0.125 [current_design]
set_wire_load_model -name 16000 -library saed32lvt_tt0p85v25c
```

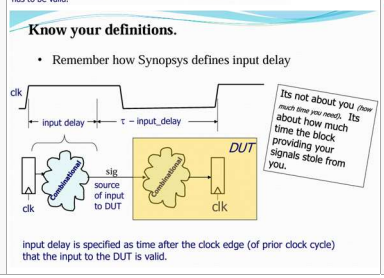
```
compile -map_effort high # Compile
ungroup -all -flatten # Optional hierarchy flattening for better timing
set_fix_hold [get_clocks clk]
compile -map_effort high -incremental

check_design > check_design.txt # Reports
report_area > report_area.txt
report_timing -path full -delay max > report_timing_max.txt  ;# setup
report_timing -path full -delay min > report_timing_min.txt  ;# hold
report_power > report_power.txt

write -format verilog -hierarchy -output MazeRunner.vg # Netlist
```

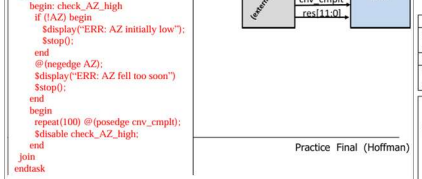
---

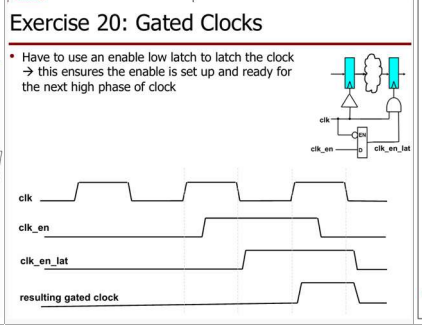
setup slack=Tclk+skew−tclk→q,max−tdata,max−tsetup

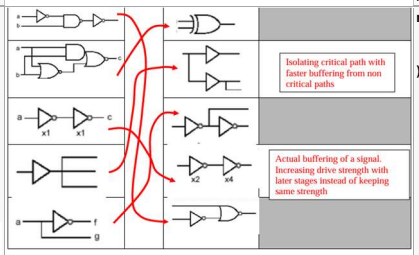
hold slack=tclk→q,min+tdata,min−thold−skew











7. **(9 pts)** Create a testbench to read and write to a sensor that uses a SPI protocol very similar to the inertial sensor we used on the project. Complete the testbench to write desired value of 0x5A to address 0x1B. Then read from address 0x1B and check that it has the correct value. You do not need to test functionality of SPI_mnrch. You do not have to show the full instantiation of SPI_mnrch (can just use ... to indicate you are connecting all the signals). Write small.

Sensor SPI Interface Protocol:	SPI_mnrch Interface:
First bit sent through SPI is the R/~W bit (1 is read, 0 is write)	**Inputs:** clk, rst_n, MISO, cmd[15:0], send_cmd
Next 7 bits are address	**Outputs:** MOSI, SCLK, SS_n, done, rd_data[15:0]
Next 8 bits are data to be written when a write, and of no consequence if it is a read.	

```
SPI_mnrch iSPI(.clk(clk), .rst_n(rst_n), ...);

initial begin
  rst_n = 0;
  clk = 0;
  send_cmd = 0;
  @(posedge clk);
  @(negedge clk);
  rst_n = 1;      // deassert reset

  cmd = 16'h1B5A;    // address with read/write_n comes as 1st byte, data as 2nd
  send_cmd = 1;
  @(negedge clk);
  send_cmd = 0;

  @(posedge done);

  cmd = 16'h9B5A;    // address with read/write_n bit.  We are reading this time
  send_cmd = 1;
  @(negedge clk);
  send_cmd = 0;

  @(posedge done);

  if (rd_data[7:0] !== 8'h5A) begin
    $display("ERR: register write/read incorrect");
    $stop();
  end

  $display("YAHOO! test passed");

end
```

```
module sv_style_example #(
  parameter int W = 16,
  parameter bit FAST_SIM = 0
)(
  input  logic clk,
  input  logic rst_n,

  input  logic       cmd_vld,
  input  logic [15:0] cmd,
  output logic       clr_cmd_vld,

  input  logic cal_done,
  input  logic mv_cmplt,
  input  logic spi_done,

  output logic strt_cal,
  output logic strt_hdng,
  output logic strt_mv,
  output logic in_cal,
  output logic send_resp,

  output logic [11:0] dsrd_hdng,
  output logic       stp_lft,
  output logic       stp_rght,

  output logic       spi_wrt,
  output logic [15:0] spi_cmd,

  input  logic [2:0]     mask,
  input  logic [W-1:0]   data [0:2],
  output logic [W+1:0]   masked_sum,
  output logic [7:0]     sum_sat,

  output logic [2:0] state_dbg
);

  typedef enum logic [2:0] {
    IDLE,
    DECODE,
    CAL_WAIT,
    HDNG_WAIT,
    MV_WAIT,
    SPI_WAIT
  } state_t;

  state_t state_q, state_d; (cont page two bottom)
```

MAZERUNNER ARCHITECTURE
Top synthesized DUT is MazeRunner.sv. Command path: RX -> UART_wrapper -> cmd_proc -> command/solver mux -> navigate. Control path: navigate creates frwrd_spd/moving/en_fusion and tells PID what maneuver is active. Feedback path: gyro SPI -> inert_intf -> inertial_integrator -> actl_hdng/hdng_rdy; IR/A2D -> sensor_intf -> openings/IR_Dtrm; IR_Math adjusts desired heading. Output path: PID -> MtrDrv -> PWM12 motor pins; piezo handles fanfare/low battery.

SYNTH CONSTRAINTS
Project synthesis target: MazeRunner level, LVT library, about 363 MHz / 2.75 ns period in the final slides/spec, with input delay 0.6 ns, output delay 0.5 ns, input drive equivalent to size-2 NAND, output load 0.1 pF, wireload 16000, max transition 0.125 ns, clock uncertainty 0.125 ns. Goal: meet timing while minimizing area. Need to write MazeRunner.vg and show at least one post-synth test passing.

WHAT TO SYNTH / EXCLUDE
Synthesize real RTL: MazeRunner, cmd_proc, maze_solve, navigate, PID/I_term/Dterm, IR_Math, MtrDrv/DutyScaleROM/PWM12, sensor_intf/A2D_intf/PWM8, inert_intf/inertial_integrator/SPI_main, UART_tx/rx/wrapper, piezo_drv, reset_synch. Exclude simulation-only/testbench models: MazeRunner_tb, tb tasks, RunnerPhysics, RemoteComm, SPI_iNEMO4 sensor model, ADC128S_FC/SPI_ADC128S A2D model, regression TBs. Exam phrase: models surround DUT; they are not part of synthesized hardware.

TOP PORTS / COMMANDS
Important ports: clk, raw active-low RST_n -> reset_synch -> rst_n; INRT SPI pins to gyro; A2D SPI pins to converter; RX/TX to BLE UART; hall_n active-low magnet -> sol_cmplt; IR_lft/cntr/rght_en; lftPWM1/2 and rghtPWM1/2; piezo/piezo_n. Commands use cmd[15:13]: 000 calibrate, 001 heading with cmd[11:0], 010 move with stop bits cmd[1:0], 011 solve using cmd[0] preference. Response/ack byte is 0xA5.

CMD_PROC
cmd_proc is command lifecycle FSM: IDLE waits cmd_rdy and clears it; EXECUTE decodes command; CAL pulses strt_cal then waits cal_done; HDNG pulses strt_hdng then waits mv_cmplt; MVNG pulses strt_mv then waits mv_cmplt; solve command lowers cmd_md so maze_solve takes over until sol_cmplt. dsrd_hdng is flopped so it persists after the command word is gone. stp_lft/stp_rght are registered from move command bits. send_resp fires when an operation completes.

MAZE_SOLVE
maze_solve is autonomous solver FSM: IDLE -> START_MOVE -> MOVE_UNTIL_OPN -> START_HEADING -> TURN. It keeps a registered dsrd_hdng initialized NORTH. While moving, stp_lft=cmd0 and stp_rght=~cmd0 act as preference/stop controls. When mv_cmplt occurs, it chooses a new heading: if both openings, cmd0 chooses left vs right; if only one side open, turn that way; if neither side open, turn around. sol_cmplt returns to IDLE.

NAVIGATE
navigate executes maneuvers and creates forward speed. States: IDLE, HDNG_INIT, HDNG, MOVE_INIT, MOVE, MV_STOP, MV_STOP_FAST. strt_hdng makes moving=1 until PID reports at_hdng after the initial heading cycle. strt_mv initializes frwrd_spd to MIN_FRWRD and ramps toward MAX_FRWRD on hdng_rdy ticks. It stops slowly on requested side-opening edge and fast when forward path closes. mv_cmplt pulses when speed reaches zero or heading completes. en_fusion turns on only above half max speed.

PID
PID computes heading correction from err = actl_hdng - dsrd_hdng, saturates to err_sat, forms P/I/D terms, sums them, divides by 8, then adds/subtracts correction around frwrd_spd: left = frwrd_spd + correction, right = frwrd_spd - correction. If moving=0, both speeds go zero. at_hdng is true when saturated error is within about +/-30. This is arithmetic-heavy, so it is a natural critical-path region.

I_TERM / DTERM
I_term integrates saturated heading error only when hdng_vld and moving are asserted; it prevents integral windup by saturating/clamping. Dterm estimates change in error using prior error samples and contributes damping, also paced by hdng_vld. These modules are inside PID and create registered arithmetic/control behavior. Good exam explanation: P reacts to current error, I accumulates persistent bias, D damps motion using derivative/change information.

IR_MATH
IR_Math adjusts desired heading using side IR readings only when en_fusion is high. If both side openings are available, correction is zero because walls are not reliable. If one side is open, use the remaining wall sensor. If both walls are present, use the left/right difference plus IR_Dtrm derivative correction. Output dsrd_hdng_adj feeds PID; in your top level it is pipelined before PID to break the IR_Math -> PID timing path.

SENSOR_INTF / A2D
sensor_intf cycles through left, center, right IR emitters and battery sampling. It waits settling time, starts A2D conversions, captures readings, subtracts calibration offsets, computes lft_opn/rght_opn/frwrd_opn using thresholds, averages vbatt, and flags batt_low. It also computes IR_Dtrm from delayed IR samples and pipelines it for timing. A2D_intf sends SPI command {2'b00, channel, 11'h000}, waits for conversion response, then exposes 12-bit result.

INERTIAL / SPI
inert_intf talks to the iNEMO gyro over SPI, performs setup, handles calibration, and produces heading readings/ready pulses. inertial_integrator integrates angular rate into actl_heading and uses calibration offset so heading is meaningful. SPI_main is a generic 16-bit SPI master: load command, assert SS_n, divide SCLK, shift MOSI from shft_reg[15], sample MISO, count 16 bits, set done, return rd_data. FAST_SIM reduces waiting/setup times in sim.

MTRDRV / DUTYSCALE / PWM
MtrDrv takes signed lft_spd/rght_spd, multiplies by scale_factor from DutyScaleROM using vbatt[9:4], saturates scaled products to signed 12-bit, then offsets to unsigned PWM duty: left final = lft_scaled + 0x800, right final = 0x800 - rght_scaled. Duty inputs are pipelined before PWM12 to break mult/saturate/PWM path. PWM12 uses a 12-bit counter, comparators, and non-overlap/dead-time for the two motor pins.

RESET / PIEZO / MODELS
reset_synch uses two flops: raw RST_n asserts reset asynchronously but deasserts synchronously as rst_n. piezo_drv is synthesized support logic: FSM/counters generate fanfare on sol_cmplt and low-battery tone on batt_low; FAST_SIM advances note durations faster. RunnerPhysics is simulation-only: it watches motor PWM, estimates wheel velocity, yaw rate, heading, wall sensors, battery, hall sensor, and feeds SPI/A2D models. RemoteComm is also TB/model infrastructure, not synthesis.

CRITICAL PATH STORY
Best project answer: the dangerous paths are arithmetic/control datapaths around IR_Math -> PID -> speed calculation -> MtrDrv scaling/saturation/PWM, not UART/SPI state machines. Your top-level added pipeline registers after IR_Math (dsrd_hdng_adj_piped), after PID speeds (lft/rght_spd_piped), and inside MtrDrv before PWM duties. Pipelining cuts long combinational chains at cost of a cycle or two of latency, which the closed-loop robot control tolerates.

FULL SYSTEM TESTS
Full-system TB should not be one giant test. Use tasks: initialize/reset, SendCmd, wait for command sent, ChkPosAck, wait_timeout, check heading/motion. Start simple: after reset, PWMs near midrail/idle and NEMO setup eventually completes. Then send calibrate: strt_cal -> in_cal -> cal_done -> positive ack. Then heading west: strt_hdng, wheel velocities split, yaw/heading changes, actl_hdng converges to dsrd_hdng. Then move: omega_sum/frwrd_spd ramps and stop conditions work.

POST-SYNTH FLOW
Synthesis demo flow: run dc_shell script, compile MazeRunner with LVT library and constraints, flatten/smash hierarchy for final area, write MazeRunner.vg, write max/min timing and area reports. In ModelSim/Questa, compile standard-cell library models, compile the gate netlist instead of RTL for synthesized modules, keep behavioral testbench/models, then run a short MazeRunner-level test. If gate sim fails, check library mapping, X reset, wrong top, compiled duplicate modules, or TB expecting old latency.

PROJECT MODULE MATCHING
Likely matching answers: cmd_proc/UART wrapper path contains UART receiver/command response logic; inertial_integrator performs sensor fusion/integrates gyro into actual heading; balance/PID control uses heading/load-like feedback to produce motor speed commands; steer/navigate produces or manages left/right speed intent; A2D_intf acquires A2D/load/IR/battery data; mtr_drv contains PWM11/PWM12-like PWM drivers and battery scaling; piezo produces charge/low-battery sound. Use the exact current module names if asked.

STATIC TIMING REPORTS
For a report, read startpoint, endpoint, path group, and path type max/min first. Data arrival time = launch clock/input delay + clk2q + net/cell delays. Data required time for max = capture edge minus setup/output delay/uncertainty. Slack max = required - arrival. For min/hold, data required is hold requirement and slack = arrival - required. In project timing, report_timing -max finds setup critical path; report_timing -min checks hold.

PRACTICE FINAL TRAPS
Functions cannot contain event/delay/timing controls. Primary input drive/load need constraints; tool cannot magically know external environment. Concurrent assertions are useful for protocols. Bottom-up compile matters mostly for large designs but can apply whenever hierarchy/complexity justify it. Modern interconnect RC is worse, not better. SDF includes gate/cell delay and interconnect flight time. FPGA is usually higher power than ASIC. APR/extraction models parasitics; it cannot remove physics.

T/F QUICK ANSWERS
Wireload model being 'incomplete' because no area impact was false for current SAED32 context if area estimates exist. Architectural optimization has bigger speed/area impact than gate tweaking. Good APR cannot eliminate parasitic capacitance. Clock gating: lower power, higher/same area, same functional latency. A fixed for loop synthesizes by unrolling. Parameters can size/select hardware; input ports cannot. Input ports can vary at runtime; parameters cannot.

SPI TB TEMPLATE
To write/read SPI sensor: reset, form write cmd = {1'b0, 7'h1B, 8'h5A}, pulse send_cmd/wrt for one clock, wait(done), maybe deassert and wait. Then form read cmd = {1'b1, 7'h1B, 8'h00}, pulse send, wait(done), assert(rd_data[7:0] == 8'h5A) else $fatal. You do not need to test SPI_mstr internals if it is provided; test transaction sequencing and response check.

AUTOZERO CHECK TEMPLATE
Task version: after reset deasserts, fork two branches. Branch A checks AZ is initially high and waits for negedge AZ; if it falls before 100 conversion-complete events, error/stop. Branch B repeat(100) @(posedge cnv_cmplt), then disables Branch A / named block. Assertion version: property sampled on conversions/reset that autozero remains high for the first 100 samples after reset. Key idea: parallel watchdog and counter, not sequential guessing.

GATE OPTIMIZATIONS
Recognize DeMorgan and bubble pushing: NAND/NOR with input/output inversions can become simpler AND/OR forms. Double inverter may be removed, or kept/upsized as buffering for fanout/timing. Logic can be factored to reduce gates, or duplicated to reduce delay. Buffer tree may replace one huge driver feeding many loads. Common subexpressions can be shared for area, but sometimes unshared for speed.

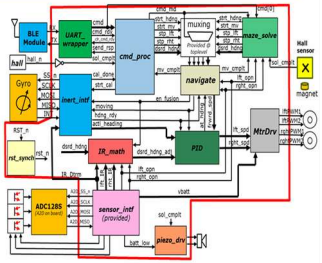
PROJECT ANSWER PHRASES
If asked 'what was your speed path and fix?' say: arithmetic-heavy heading/speed/control path; solved with LVT cells plus pipeline registers at IR_Math->PID, PID->MtrDrv, and duty->PWM boundary; then verified with max/min timing and post-synth MazeRunner-level sim. If asked 'why extra latency okay?' say closed-loop control is paced by sensor/heading updates and tolerates a few cycles while meeting 363 MHz timing.

CMD_PROC FSM
Purpose: UART command lifecycle controller. IDLE waits for cmd_rdy from UART_wrapper; when seen, assert clr_cmd_rdy and go EXECUTE. EXECUTE decodes cmd[15:13]: 000=calibrate -> pulse strt_cal, go CAL; 001=heading -> flop dsrd_hdng=cmd[11:0], pulse strt_hdng, go HDNG; 010=move -> flop stp_lft=cmd[1], stp_rght=cmd[0], pulse strt_mv, go MVNG; 011=solve -> cmd_md=0 so maze_solve controls mux until sol_cmplt. CAL holds in_cal=1 until cal_done, then send_resp=1 and IDLE. HDNG/MVNG wait mv_cmplt, then send_resp=1 and IDLE. Key hardware idea: start signals are one-cycle pulses; dsrd_hdng/stop bits are flops because command bus is temporary; send_resp triggers positive ack 0xA5.

Toplevel Digital of our Design

The blocks outlined in red are pure digital blocks, and will be coded with the intent of being synthesized via Synopsys to our standard cell library. For practical purposes we will also map that logic to a DE0 FPGA board so we can run the demos.

You Must have a block called **MazeRunner.sv** which is top level of what will be the synthesized DUT. A shell is provided to ensure we all have the same interface.



SPI_MAIN FSM
Purpose: reusable 16-bit SPI master = FSM + SCLK divider + shift register + bit counter. IDLE: SS_n=1, wait wrt; on wrt, clear done, load shft_reg=wt_data, clear bit_cnt, reset SCLK_div, go READ. READ: SS_n=0, start SCLK; sample MISO at SCLK_div==01111; when SCLK_div==11111 go TRANSFER. TRANSFER: SS_n=0, MOSI=shft_reg[15] sends MSB-first; sample MISO at sample phase; at shift phase do shft_reg={shft_reg[14:0],MISO_smpl}; increment bit_cnt; after 16 bits go FINISH. FINISH: SS_n=1, set done=1, rd_data=shft_reg, return IDLE. Key exam phrase: wrt is start-transaction pulse, done means rd_data valid.

CONCURRENT ASSERTIONS / PROPERTY BLOCKS
Immediate assertion checks now: assert(x==y) else $fatal. Concurrent assertion checks behavior across clock cycles. Syntax: property name; @(posedge clk) disable iff(!rst_n) antecedent |-> consequent; endproperty; assert property(name). |-> means same-cycle implication; |=> means next-cycle implication. ##N means wait N clocks; ##[1:3] means wait 1 to 3 clocks; sig[*N] means sig holds/repeats N cycles. Use for protocols: request followed by ack, start held until done, no illegal FSM transition, autozero high for first 100 conversions. // after a command starts, response must eventually happen property p_cmd_resp; @(posedge clk) disable iff(!rst_n) send_cmd |-> ##[1:2000] send_resp; endproperty assert property(p_cmd_resp) else $fatal("command timeout");

INERT_INTF AROUND SPI
inert_intf is gyro-specific control around generic SPI_main. It configures iNEMO with init SPI writes, double-flops INT, waits for INT rising edge = new gyro data ready, sends read cmd 16'hA600 for yaw low, waits done and stores yaw_low=inert_data[7:0], sends 16'hA700 for yaw high, waits done and stores yaw_high, forms yaw_rt={yaw_high,yaw_low}, pulses vld, then inertial_integrator subtracts calibration/bias and integrates angular rate into actl_heading. strt_cal starts calibration; cal_done returns to cmd_proc; rdy means fresh heading update.

```
logic [15:0] cmd_q;
logic load_cmd;
logic load_hdng;
logic load_move;
logic [15:0] timeout_limit;

assign state_dbg = state_q;

function automatic logic [7:0] sat8(input logic [9:0] x);
  if (x > 10'd255)
    sat8 = 8'hFF;
  else
    sat8 = x[7:0];
endfunction
assign sum_sat = sat8(masked_sum[9:0]);

generate
  if (FAST_SIM)
    assign timeout_limit = 16'd20;
  else
    assign timeout_limit = 16'd1000;
```

PIEZO_DRV
Piezo = piezoelectric buzzer. piezo_drv does not directly read hall_n; top level makes sol_cmplt=~hall_n, then passes fanfare=sol_cmplt. FSM states: IDLE, BATTERY_LOW, FANFARE. IDLE waits for batt_low or fanfare. BATTERY_LOW repeats G6,C7,E7 while batt_low=1. FANFARE plays charge sequence G6,C7,E7,G7,E7,G7 once, then IDLE or BATTERY_LOW if batt_low still high. cyc_counter sets square-wave pitch; dur_counter sets note length; idx selects current note. piezo is square wave when active, 0 in IDLE; piezo_n=~piezo.

```
  endgenerate

  always_comb begin
    masked_sum = '0;

    for (int i = 0; i < 3; i++) begin
      if (mask[i])
        masked_sum = masked_sum + {{2{1'b0}}, data[i]};
    end
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      state_q   <= IDLE;
      cmd_q     <= 16'h0000;
      dsrd_hdng <= 12'h000;
      stp_lft   <= 1'b0;
      stp_rght  <= 1'b0;
    end else begin
      state_q <= state_d;\
      if (load_cmd)
```

IR_MATH -> PID -> MTRDRV FLOW
IR_Math is combinational wall-following correction: if en_fusion=0, dsrd_hdng_adj=dsrd_hdng. If both walls open, no correction; if one wall exists, compare that IR reading to NOM_IR; if both walls exist, use left-right IR difference. Add P wall term + IR_Dtrm derivative term to nudge desired heading. PID then computes err=actl_hdng-dsrd_hdng_adj, saturates error, forms P/I/D, divides correction by 8, and outputs lft_spd=frwrd_spd+corr, rght_spd=frwrd_spd-corr when moving, else both zero. MtrDrv scales signed speeds using DutyScaleROM(vbatt[9:4]), saturates, converts to PWM duty around 0x800 neutral, and drives PWM12. Critical path risk: IR_Math/PID/MtrDrv arithmetic; fix by pipelining between stages cmd_q <= cmd;

```
    if (load_hdng)
      dsrd_hdng <= cmd_q[11:0];

    if (load_move) begin
      stp_lft  <= cmd_q[1];
      stp_rght <= cmd_q[0];
    end
  end
end
always_comb begin
  state_d     = state_q;
  clr_cmd_vld = 1'b0;
  strt_cal    = 1'b0;
  strt_hdng   = 1'b0;
  strt_mv     = 1'b0;
  in_cal      = 1'b0;
```